**ACS-2947-002/003**

**Lab 4**

**Due by Tuesday, February 11, 11:59 PM**

**Instructions**

- The goal of this lab is to understand the implementation of the Linked Positional List and learn to use its different methods.
- Submit your `.java` files (together in a `Lab4.zip` file) via Nexus.
- Include your name and student number as a comment in every file.

**Task**

1. Implement a `LinkedPositionalList` using the `PositionalList` and `Position` interfaces provided. Note that the `PositionalList` interface provided for this lab does not extend the `Iterable` interface. You may choose to use the partial implementation (`LinkedPositionalList`) provided for you. You will need to add the following:
   - Private methods `validate` and `position`
   - `addBetween`, `addBefore`, `addAfter`, `set`, and `remove`

2. In a driver class `Lab4_Driver`, create a positional list that stores the alphabets A to O in **descending** order. Display the list contents and cursor elements after each of the following steps:
   a. Declare a variable named `cursor` and assign it to the first element in the list.
   b. Move the cursor to the 9th position in the list, i.e., the cursor should hold the position that references letter G.
   c. Add the letter Q before the cursor and the letter P after the cursor.
   d. Move the cursor backwards 5 positions.
   e. Change the letter after the cursor to Y. Delete the item before the cursor.
   f. Move the cursor forward 6 positions and change the letter in this position to S.
   g. Add the letter A in the first position
   h. Reset your cursor to the first position and traverse through your list to delete all positions with an element whose ASCII value is an odd number.
   i. Add letter Z at the end of the list. **Note:** you must not use the `addLast` method for this question.

Create a method that displays the output of a – i in the format shown below.

**Sample output for each step (a – i):**

```
Q2a:
Cursor is at: O
LPL contents: [O, N, M, L, K, J, I, H, G, F, E, D, C, B, A]

Q2b:
Cursor is at: ...
LPL contents: ...
...
```

3. Display the number of entries remaining in the list.

**Submission**

Submit your **`Lab4.zip`** file that include all the following files (`LinkedPositionalList.java, PositionalList, Position.java, Lab4_Driver.java`) via **Nexus**.

**EXTRA WORK: Do not submit**

Create a class called `DynamicArray` which holds a generic array as a private member. Your class should grow the array if it runs out of space and shrink the array if it is less than half full. It must include public methods to add/remove objects to/from your array, as well as a method that resizes the array accordingly.

*Use the ArrayList implementation from your notes as a guide.*

Create a driver class to illustrate how the capacity of your array changes as objects are added and removed.